

Федеральное государственное автономное образовательное учреждение высшего образования  
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

## **Лабораторная работа №6**

«Численное решение обыкновенных  
дифференциальных уравнений»

по курсу «Вычислительная математика»

Вариант 19

Выполнил:

Якименко Владислав Игоревич, Р3213

Проверила:

Малышева Татьяна Алексеевна

# Содержание

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы и постановка задачи</b>                          | <b>1</b>  |
| 1.1      | Задача Коши . . . . .                                           | 1         |
| 1.2      | Вариант задания . . . . .                                       | 1         |
| 1.3      | Уравнения, предлагаемые программой . . . . .                    | 1         |
| <b>2</b> | <b>Описание алгоритма решения задачи</b>                        | <b>2</b>  |
| 2.1      | Метод конечных разностей . . . . .                              | 2         |
| 2.2      | Общая схема работы программы . . . . .                          | 2         |
| <b>3</b> | <b>Рабочие формулы используемых методов</b>                     | <b>4</b>  |
| 3.1      | Метод Эйлера (метод 1) . . . . .                                | 4         |
| 3.2      | Метод Рунге–Кутты 4-го порядка (метод 3) . . . . .              | 4         |
| 3.3      | Метод Адамса, схема предиктор-корректор (метод 4) . . . . .     | 4         |
| 3.4      | Оценка погрешности одношаговых методов: правило Рунге . . . . . | 5         |
| 3.5      | Оценка погрешности многошагового метода . . . . .               | 5         |
| <b>4</b> | <b>Программная реализация</b>                                   | <b>6</b>  |
| 4.1      | Структура проекта . . . . .                                     | 6         |
| 4.2      | Ввод исходных данных . . . . .                                  | 6         |
| 4.3      | Листинг программы . . . . .                                     | 6         |
| <b>5</b> | <b>Результаты выполнения программы</b>                          | <b>14</b> |
| 5.1      | Задача варианта . . . . .                                       | 14        |
| 5.2      | Правило Рунге . . . . .                                         | 15        |
| 5.3      | Порядок сходимости . . . . .                                    | 15        |
| 5.4      | Другие наборы исходных данных . . . . .                         | 16        |
| 5.5      | Проверка программы . . . . .                                    | 17        |
| <b>6</b> | <b>Графики точного и приближённого решений</b>                  | <b>18</b> |
| <b>7</b> | <b>Скриншоты результатов работы программы</b>                   | <b>21</b> |
| <b>8</b> | <b>Вывод</b>                                                    | <b>25</b> |

# 1 Цель работы и постановка задачи

**Цель работы:** решить задачу Коши для обыкновенного дифференциального уравнения первого порядка численными методами, сравнить одношаговые и многошаговые схемы и оценить погрешность полученных решений.

## 1.1 Задача Коши

Требуется найти функцию  $y = y(x)$ , удовлетворяющую уравнению первого порядка, разрешённому относительно производной,

$$y' = f(x, y), \quad x \in [x_0, x_n],$$

и начальному условию

$$y(x_0) = y_0.$$

Решить задачу численно — значит для последовательности узлов  $x_0 < x_1 < \dots < x_n$  с постоянным шагом  $h = x_{i+1} - x_i$  приближённо вычислить значения  $y_1, \dots, y_n$  сеточной функции, приближающей точное решение в этих узлах.

Достаточные условия существования и единственности даёт теорема Коши: если  $f(x, y)$  непрерывна в замкнутой области  $\bar{D} = \{(x, y) : |x - x_0| \leq a, |y - y_0| \leq b\}$ , решение существует; если, кроме того,  $f$  удовлетворяет условию Липшица по  $y$

$$|f(x, y_1) - f(x, y_2)| \leq k|y_1 - y_2|, \quad k = \text{const} > 0,$$

то решение единственно. Геометрически задача Коши — это поиск интегральной кривой, проходящей через заданную начальную точку  $M_0(x_0, y_0)$ .

## 1.2 Вариант задания

Номер варианта — 19. По таблице 1 задания варианту соответствует набор методов **1, 3, 4**:

- одношаговые: **метод Эйлера** (метод 1) и **метод Рунге–Кутты 4-го порядка** (метод 3);
- многошаговый: **метод Адамса** в схеме предиктор-корректор (метод 4).

Точность одношаговых методов оценивается по **правилу Рунге**, точность многошагового — по точному решению:  $\varepsilon = \max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|$ .

## 1.3 Уравнения, предлагаемые программой

Задание требует предложить пользователю не менее трёх уравнений. Программа предлагает шесть; для каждого известно аналитическое решение задачи Коши, поэтому погрешность можно измерить в каждом узле.

Таблица 1: Уравнения и точные решения задачи Коши

| Идентификатор | $y' = f(x, y)$    | Точное решение                                                              |
|---------------|-------------------|-----------------------------------------------------------------------------|
| linear        | $y$               | $y = y_0 e^{x-x_0}$                                                         |
| sum           | $x + y$           | $y = (y_0 + x_0 + 1)e^{x-x_0} - x - 1$                                      |
| square        | $y - x^2 + 1$     | $y = (x + 1)^2 + Ce^x, C = (y_0 - (x_0 + 1)^2)e^{-x_0}$                     |
| bernoulli     | $y + (1 + x)y^2$  | $y = \frac{1}{Ce^{-x} - x}, C = e^{x_0} \left( \frac{1}{y_0} + x_0 \right)$ |
| gauss         | $-2xy$            | $y = y_0 e^{x_0^2 - x^2}$                                                   |
| stiff         | $-5y + 5x^2 + 2x$ | $y = x^2 + Ce^{-5x}, C = (y_0 - x_0^2)e^{5x_0}$                             |

Уравнение **bernoulli** совпадает с примером 1 лекции: при  $y(1) = -1$  постоянная  $C = e\left(\frac{1}{-1} + 1\right) = 0$ , и точное решение принимает вид  $y = -\frac{1}{x}$ . Это позволяет сверить работу программы с таблицами, приведёнными в лекции, поэтому именно эта задача выбрана в качестве основного демонстрационного набора:

$$y' = y + (1 + x)y^2, \quad y(1) = -1, \quad x \in [1; 1.5], \quad h = 0.1, \quad \varepsilon = 10^{-6}.$$

## 2 Описание алгоритма решения задачи

### 2.1 Метод конечных разностей

Все реализованные методы основаны на методе конечных разностей. Область непрерывного изменения аргумента — отрезок  $[x_0, x_n]$  — заменяется дискретным множеством узлов  $x_i = x_0 + ih$ , образующих разностную сетку. Искомая функция непрерывного аргумента заменяется сеточной функцией  $y_i$ , а дифференциальное уравнение — разностным уравнением относительно  $y_i$ ; входящая в уравнение производная заменяется конечно-разностным отношением. Такая замена называется разностной аппроксимацией на сетке.

### 2.2 Общая схема работы программы

1. Пользователь выбирает уравнение  $y' = f(x, y)$  из предложенного списка.
2. С клавиатуры (либо из файла) вводятся  $x_0$ ,  $y_0 = y(x_0)$ , правая граница  $x_n$ , шаг  $h$  и точность  $\varepsilon$ .
3. Входные данные проверяются:  $x_n > x_0$ ,  $h > 0$ ,  $h \leq x_n - x_0$ ,  $0 < \varepsilon < 1$ , число узлов не превышает предела, правая часть определена в начальной точке.
4. Строится сетка  $x_i = x_0 + ih$ ,  $i = 0, \dots, n$ ; вычисляется точное решение  $y_{i \text{ точн}}$  в узлах.

5. Задача решается тремя методами варианта. Метод Адамса перед началом счёта получает  $y_1, y_2, y_3$  методом Рунге–Кутта.
6. Составляется общая таблица приближённых значений и погрешностей.
7. Для одношаговых методов выполняется контроль точности по правилу Рунге: задача решается с шагами  $h$  и  $h/2$ , вычисляется  $R$ ; пока  $R > \varepsilon$ , шаг делится пополам.
8. Для метода Адамса вычисляется  $\varepsilon = \max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|$ .
9. Строятся графики точного и приближённых решений (разными цветами) и график погрешности; результаты выводятся в консоль, в файл и в браузер.

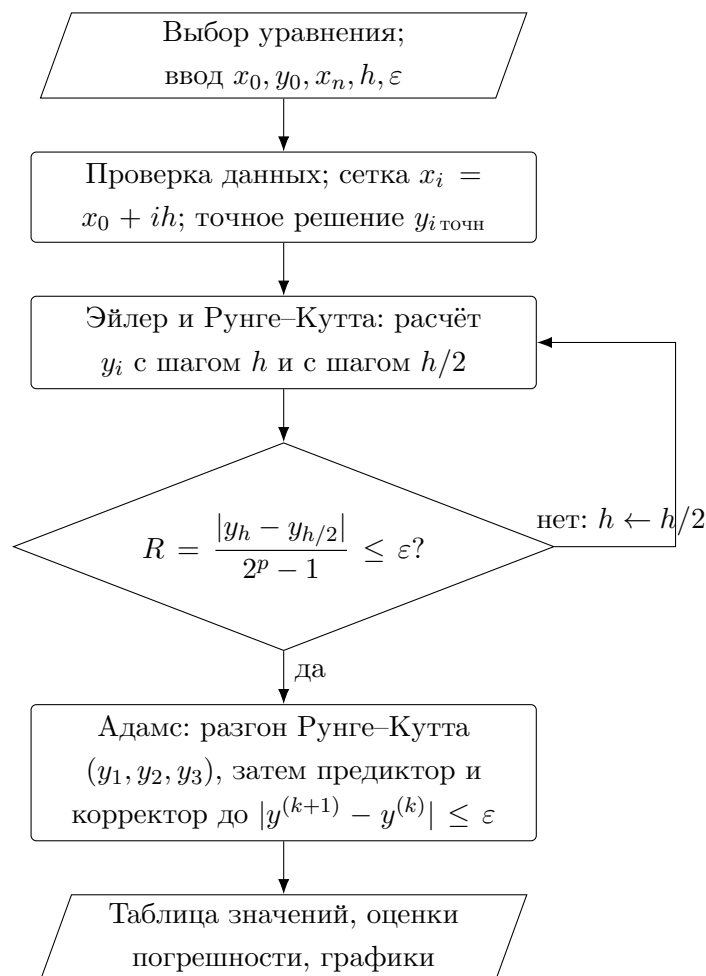


Рис. 1: Блок-схема алгоритма программы

## 3 Рабочие формулы используемых методов

### 3.1 Метод Эйлера (метод 1)

Метод основан на разложении решения в ряд Тейлора в окрестности узла  $x_i$  с отбрасыванием членов, содержащих производные второго и более высоких порядков:

$$Y(x_i + h) = Y(x_i) + Y'(x_i)h + O(h^2), \quad Y'(x_i) = f(x_i, y_i) \approx \frac{y_{i+1} - y_i}{h}.$$

Отсюда получается расчётная формула

$$y_{i+1} = y_i + hf(x_i, y_i), \quad i = 0, 1, \dots, n-1, \quad (1)$$

$$y_0 = Y_0. \quad (2)$$

Геометрически на каждом шаге приращение решения заменяется приращением ординаты касательной, поэтому после первого шага счёт продолжается уже по другой интегральной кривой. Метод одношаговый, порядок точности  $p = 1$ , суммарная погрешность  $\delta_n = O(h)$ .

### 3.2 Метод Рунге–Кутты 4-го порядка (метод 3)

Идея методов Рунге–Кутты: не использовать частные производные  $f(x, y)$ , а вычислять саму правую часть в нескольких точках внутри шага. Для наиболее распространённого метода 4-го порядка:

$$y_{i+1} = y_i + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad (3)$$

$$\begin{aligned} k_1 &= hf(x_i, y_i), & k_2 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right), \\ k_3 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right), & k_4 &= hf(x_i + h, y_i + k_3). \end{aligned} \quad (4)$$

Метод требует четырёхкратного вычисления правой части на каждом шаге, но имеет суммарную погрешность  $\delta_n = O(h^4)$ , что позволяет считать с существенно большим шагом, чем метод Эйлера. Метод одношаговый: значение  $y_{i+1}$  строится только по  $y_i$ , поэтому шаг можно менять в любой момент.

### 3.3 Метод Адамса, схема предиктор-корректор (метод 4)

Многошаговые методы получаются интегрированием уравнения на отрезке  $[x_i, x_{i+1}]$  с заменой правой части интерполяционным многочленом  $P_{k-1}$ , построенным по значениям  $f_{i-k+1}, \dots, f_i$ :

$$y_{i+1} = y_i + \int_{x_i}^{x_{i+1}} P_{k-1}(x) dx.$$

При  $k = 4$  и постоянном шаге многочлен Ньютона строится по конечным разностям правой части

$$\Delta f_i = f_i - f_{i-1}, \quad \Delta^2 f_i = f_i - 2f_{i-1} + f_{i-2}, \quad \Delta^3 f_i = f_i - 3f_{i-1} + 3f_{i-2} - f_{i-3},$$

и разностная схема метода Адамса 4-го порядка принимает вид

$$y_{i+1} = y_i + h \left( f_i + \frac{1}{2} \Delta f_i + \frac{5}{12} \Delta^2 f_i + \frac{3}{8} \Delta^3 f_i \right). \quad (5)$$

Раскрыв разности, получаем явную формулу с числовыми коэффициентами. Именно она используется в программе как **этап предиктора**:

$$y_{i+1}^{\text{прогн}} = y_i + \frac{h}{24} (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}). \quad (6)$$

**Этап корректора** использует неявную формулу, в правую часть которой входит уже уточняемое значение через  $f_{i+1} = f(x_{i+1}, y_{i+1})$ :

$$y_{i+1}^{(s+1)} = y_i + \frac{h}{24} \left( 9f(x_{i+1}, y_{i+1}^{(s)}) + 19f_i - 5f_{i-1} + f_{i-2} \right). \quad (7)$$

Итерации корректора продолжаются, пока  $|y_{i+1}^{(s+1)} - y_{i+1}^{(s)}| > \varepsilon$ ; при выполнении этого неравенства выполняется переход к следующему узлу. Начальным приближением  $y_{i+1}^{(0)}$  служит предиктор (6).

Формулы (6) и (7) требуют значений в четырёх предыдущих узлах, поэтому счёт может быть начат только с  $y_4$ . Значения  $y_1, y_2, y_3$  получены методом Рунге–Кутты 4-го порядка,  $y_0$  задано начальным условием. Суммарная погрешность метода  $\delta_n = O(h^4)$ ; при этом на каждом шаге правая часть вычисляется всего один-два раза против четырёх у метода Рунге–Кутты.

*Замечание.* В лекции формула (17) записана в виде  $y_{i+1} = y_i + hf_i + \frac{h^2}{2} \Delta f_i + \frac{5h^3}{12} \Delta^2 f_i + \frac{3h^4}{8} \Delta^3 f_i$ . Конечные разности  $\Delta^k f_i$  уже безразмерны относительно  $h$ , поэтому множителем при всех слагаемых должно быть  $h$  в первой степени — как в (5). Именно в такой записи (5) раскрывается ровно в (6), что подтверждено автоматическим тестом.

### 3.4 Оценка погрешности одношаговых методов: правило Рунге

Для метода порядка  $p$  решение вычисляется дважды — с шагом  $h$  и с шагом  $h/2$ , — после чего погрешность оценивается как

$$R = \frac{y_h - y_{h/2}}{2^p - 1}, \quad (8)$$

где  $y_h$  и  $y_{h/2}$  — решения в одном и том же узле. В программе контроль выполняется на всём интервале:  $R$  вычисляется во всех общих узлах двух сеток и берётся максимум по модулю. Если  $|R| \leq \varepsilon$ , точность считается достигнутой; иначе шаг делится пополам и расчёт повторяется. Для метода Эйлера  $p = 1$ , для метода Рунге–Кутты  $p = 4$ .

### 3.5 Оценка погрешности многошагового метода

Для метода Адамса, как требует задание, используется точное решение:

$$\varepsilon = \max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|. \quad (9)$$

Такая оценка применима именно потому, что все предлагаемые программой уравнения имеют аналитическое решение. Она же вычисляется и для одношаговых методов — это позволяет сравнить фактическую погрешность с оценкой (8).

## 4 Программная реализация

### 4.1 Структура проекта

Программа написана на JavaScript для Node.js 20+, сторонних зависимостей нет. Численные методы вынесены в отдельный модуль `src/ode.js`: каждый метод — это самостоятельная функция с одинаковой сигнатурой  $(f, x_0, y_0, x_n, h)$ , а список `METHODS` связывает метод с его порядком точности и способом оценки погрешности.

Таблица 2: Состав проекта

| Путь                              | Содержимое                                                 |
|-----------------------------------|------------------------------------------------------------|
| <code>src/ode.js</code>           | Уравнения, проверка данных, все методы, оценка погрешности |
| <code>src/io.js</code>            | Чтение задачи из файла и форматирование таблиц             |
| <code>src/index.js</code>         | Консольное меню, ввод с клавиатуры, параметры запуска      |
| <code>src/plotting-core.js</code> | Построение SVG-графиков решений и погрешности              |
| <code>src/gui-server.js</code>    | Локальный сервер веб-интерфейса                            |
| <code>src/report.js</code>        | Пересчёт результатов, текстовых файлов и графиков          |
| <code>src/ode.test.js</code>      | 32 автоматических теста                                    |
| <code>web/</code>                 | Интерфейс браузера                                         |
| <code>examples/</code>            | 5 корректных наборов, текстовый набор, 4 ошибочных         |
| <code>output/, docs/</code>       | Результаты запусков, графики, снимки экрана                |

### 4.2 Ввод исходных данных

Данные задаются тремя способами:

- с клавиатуры — интерактивное меню выбирает уравнение и запрашивает  $x_0, y_0, x_n, h, \varepsilon$  (десятичная запятая допускается);
- из файла — JSON либо текст вида «имя = значение»;
- параметрами командной строки `--equation, --x0, --y0, --xn, --h, --eps`.

Тот же расчётный модуль используется веб-интерфейсом, поэтому консоль и браузер дают одинаковые числа.

### 4.3 Листинг программы

Ниже приведён расчётный модуль `src/ode.js` целиком: описание уравнений с точными решениями, проверка входных данных, метод Эйлера, метод Рунге–Кутты



4-го порядка, метод Адамса в схеме предиктор-корректор, правило Рунге и оценка по точному решению.

```
1 import { parseNumber } from "./utils.js";
2
3 // Каждое уравнение задаётся правой частью f(x, y) и точным решением задачи Коши.
4 // exact(x0, y0) возвращает функцию y(x) либо null, если условие недопустимо.
5 export const EQUATIONS = [
6   {
7     id: "linear",
8     label: "y' = y",
9     solutionLabel: "y = .y·e^(-xx)",
10    f: (x, y) => y,
11    exact: (x0, y0) => x => y0 * Math.exp(x - x0),
12  },
13  {
14    id: "sum",
15    label: "y' = x + y",
16    solutionLabel: "y = (.y+.x+1)·e^(-xx) - x - 1",
17    f: (x, y) => x + y,
18    exact: (x0, y0) => x => (y0 + x0 + 1) * Math.exp(x - x0) - x - 1,
19  },
20  {
21    id: "square",
22    label: "y' = y - x2 + 1",
23    solutionLabel: "y = (x+1)2 + C·e^-x, C = (-y(,x+1)2)·e^-(,x)",
24    f: (x, y) => y - x * x + 1,
25    exact: (x0, y0) => {
26      const c = (y0 - (x0 + 1) ** 2) * Math.exp(-x0);
27      return x => (x + 1) ** 2 + c * Math.exp(x);
28    },
29  },
30  {
31    id: "bernoulli",
32    label: "y' = y + (1+x)·y2",
33    solutionLabel: "y = 1/(C·e^-(x) - x), C = e^-(,x)·(1/,y + ,x)",
34    note: "Пример 1 из лекции: y(1) = -1 на [1; 1.5] даёт y = -1/x.",
35    f: (x, y) => y + (1 + x) * y * y,
36    exact: (x0, y0) => {
37      if (y0 === 0) return x => 0;
38      const c = Math.exp(x0) * (1 / y0 + x0);
39      return x => {
40        const denominator = c * Math.exp(-x) - x;
41        return denominator === 0 ? Number.NaN : 1 / denominator;
42      };
43    },
44  },
45  {
46    id: "gauss",
47    label: "y' = -2·x·y",
48    solutionLabel: "y = .y·e^-(x2-x2)",
```

```

49   f: (x, y) => -2 * x * y,
50   exact: (x0, y0) => x => y0 * Math.exp(x0 * x0 - x * x),
51 },
52 {
53   id: "stiff",
54   label: "y' = -5y + 5x2 + 2x",
55   solutionLabel: "y = x2 + C·e-(5x), C = (·,·yx2)·e(5·x)",
56   note: "Быстро затухающая составляющая: явный метод Эйлера расходится при h > 0.4.",
57   f: (x, y) => -5 * y + 5 * x * x + 2 * x,
58   exact: (x0, y0) => {
59     const c = (y0 - x0 * x0) * Math.exp(5 * x0);
60     return x => x * x + c * Math.exp(-5 * x);
61   },
62 },
63 ];
64
65 export const MAX_NODES = 200000;
66 export const MAX_REFINEMENTS = 20;
67
68 // Принимает идентификатор либо само уравнение, поэтому validateProblem идемпотентна.
69 export function findEquation(id) {
70   if (id && typeof id === "object") id = id.id;
71   const equation = EQUATIONS.find(item => item.id === id);
72   if (!equation) throw new ErrorНеизвестное(`уравнение «${id}». Доступны: ${EQUATIONS.
73     map(e => e.id).join(", ")}.`);
74   return equation;
75 }
76
77 export function validateProblem(input) {
78   if (!input || typeof input !== "object") throw new Error("Требуется описание задачи
79     Коши.");
80   const equation = findEquation(input.equation ?? input.equationId);
81   const x0 = parseNumber(input.x0, ".x");
82   const y0 = parseNumber(input.y0, ".y");
83   const xn = parseNumber(input.xn, ".xn");
84   const h = parseNumber(input.h, "Шаг h");
85   const epsilon = parseNumber(input.epsilon ?? 1e-6, "Точность ε");
86   if (xn <= x0) throw new Error("Правая граница xn должна быть больше .x.");
87   if (h <= 0) throw new Error("Шаг h должен быть положительным.");
88   if (h > xn - x0) throw new Error("Шаг h не должен превышать длину интервала.");
89   if (epsilon <= 0) throw new Error("Точность ε должна быть положительной.");
90   if (epsilon >= 1) throw new Error("Точность ε должна быть меньше единицы.");
91   if ((xn - x0) / h > MAX_NODES) throw new ErrorСлишком(`мелкий шаг: узлов больше ${
92     MAX_NODES}`);
93   const problem = { equation, x0, y0, xn, h, epsilon };
94   const first = equation.f(x0, y0);
95   if (!Number.isFinite(first)) throw new Error("Правая часть не определена в начальной
96     точке.");
97   return problem;
98 }

```

```

95
96 // Узлы строятся от x0 с шагом h; последний шаг не выходит за xn.
97 export function gridNodes(x0, xn, h) {
98     const count = Math.max(1, Math.round((xn - x0) / h));
99     const nodes = Array.from({ length: count + 1 }, (_, i) => x0 + i * h);
100     nodes[count] = Math.min(nodes[count], xn);
101     return nodes;
102 }
103
104 function guard(value, method, x) {
105     if (!Number.isFinite(value)) {
106         throw new Error(`${method}: решение ушло в бесконечность около x = ${x.toPrecision(6)}`);
107     }
108     return value;
109 }
110
111 // ----- Одношаговые методы -----
112
113 // Метод Эйлера:  $y_{i+1} = y_i + h \cdot f(x_i, y_i)$ . Порядок точности  $p = 1$ .
114 export function euler(f, x0, y0, xn, h) {
115     const nodes = gridNodes(x0, xn, h);
116     const values = [y0];
117     for (let i = 1; i < nodes.length; i++) {
118         const step = nodes[i] - nodes[i - 1];
119         values.push(guard(values[i - 1] + step * f(nodes[i - 1], values[i - 1]), "Метод Эйлера", nodes[i]));
120     }
121     return { nodes, values };
122 }
123
124 // Усовершенствованный метод Эйлера метод( Эйлера с пересчётом),  $p = 2$ .
125 // Реализован для сравнения; в вариант 19 не входит.
126 export function improvedEuler(f, x0, y0, xn, h) {
127     const nodes = gridNodes(x0, xn, h);
128     const values = [y0];
129     for (let i = 1; i < nodes.length; i++) {
130         const step = nodes[i] - nodes[i - 1];
131         const slope = f(nodes[i - 1], values[i - 1]);
132         const predictor = values[i - 1] + step * slope;
133         const next = values[i - 1] + step / 2 * (slope + f(nodes[i], predictor));
134         values.push(guard(next, "Усовершенствованный метод Эйлера", nodes[i]));
135     }
136     return { nodes, values };
137 }
138
139 // Метод РунгеКутты- го4- порядка,  $p = 4$ .
140 export function rungeKutta4(f, x0, y0, xn, h) {
141     const nodes = gridNodes(x0, xn, h);
142     const values = [y0];

```

```

143   for (let i = 1; i < nodes.length; i++) {
144       const step = nodes[i] - nodes[i - 1];
145       const x = nodes[i - 1], y = values[i - 1];
146       const k1 = step * f(x, y);
147       const k2 = step * f(x + step / 2, y + k1 / 2);
148       const k3 = step * f(x + step / 2, y + k2 / 2);
149       const k4 = step * f(x + step, y + k3);
150       values.push(guard(y + (k1 + 2 * k2 + 2 * k3 + k4) / 6, "Метод РунгеКутта-", nodes[i])
151       );
152   }
153   return { nodes, values };
154 }
155 // ----- Многошаговый метод -----
156
157 // Метод Адамса го4- порядка в схеме предикторкорректор-.
158 // Предиктор АдамсБашфорт(-):  $y_{i+1} = y_i + h/24 \cdot (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$ .
159 // Корректор АдамсМоултон(-):  $y_{i+1} = y_i + h/24 \cdot (9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2})$ ,
160 // итерируется до  $|y^{(k+1)} - y^{(k)}| \leq \epsilon$ . Разгон - методом РунгеКутта-.
161 export const ADAMS_MAX_ITERATIONS = 20;
162
163 export function adams(f, x0, y0, xn, h, epsilon = 1e-9) {
164     const nodes = gridNodes(x0, xn, h);
165     if (nodes.length < 5) {
166         throw new Error("Метод Адамса требует не менее четырёх шагов: возьмите  $h \leq (-, xnx) / 4.$ ");
167     }
168     const start = rungeKutta4(f, x0, y0, nodes[3], h);
169     const values = start.values.slice(0, 4);
170     const slopes = values.map((y, i) => f(nodes[i], y));
171     const corrections = [0, 0, 0, 0];
172     for (let i = 3; i < nodes.length - 1; i++) {
173         const step = nodes[i + 1] - nodes[i];
174         let y = values[i] + step / 24 * (55 * slopes[i] - 59 * slopes[i - 1] + 37 * slopes[i - 2] - 9 * slopes[i - 3]);
175         guard(y, "Метод Адамса", nodes[i + 1]);
176         let iterations = 0, delta = Infinity;
177         while (iterations < ADAMS_MAX_ITERATIONS && delta > epsilon) {
178             const corrected = values[i] + step / 24
179                 * (9 * f(nodes[i + 1], y) + 19 * slopes[i] - 5 * slopes[i - 1] + slopes[i - 2]);
180             guard(corrected, "Метод Адамса", nodes[i + 1]);
181             delta = Math.abs(corrected - y);
182             y = corrected;
183             iterations++;
184         }
185         if (delta > epsilon) {
186             throw new ErrorМетод(` Адамса: корректор не сошёлся за ${ADAMS_MAX_ITERATIONS}
            итераций при  $x = ${nodes[i + 1].toPrecision(6)}$ . Уменьшите шаг  $h$ .`);

```

```

187     }
188     values.push(y);
189     slopes.push(f(nodes[i + 1], y));
190     corrections.push(iterations);
191 }
192 return { nodes, values, corrections, startedBy: "РунгеКутта-го4- порядка" };
193 }
194
195 // Конечные разности правой части:  $\Delta f_i$ ,  $\Delta^2 f_i$ ,  $\Delta^3 f_i$  - форма (17) из лекции.
196 export function backwardDifferences(slopes, i) {
197     const [f3, f2, f1, f0] = [slopes[i - 3], slopes[i - 2], slopes[i - 1], slopes[i]];
198     return { d1: f0 - f1, d2: f0 - 2 * f1 + f2, d3: f0 - 3 * f1 + 3 * f2 - f3 };
199 }
200
201 // ----- Оценка погрешности -----
202
203 // Правило Рунге:  $R = (y_h - y_{h/2}) / (2^p - 1)$  в общих узлах двух сеток.
204 // Узлы сопоставляются по значению  $x$ , поэтому неполный последний шаг не мешает.
205 export function rungeError(coarse, fine, order) {
206     const denominator = 2 ** order - 1;
207     const tolerance = Math.max(1e-9 * Math.abs(coarse.nodes.at(-1) - coarse.nodes[0]),
208         Number.MIN_VALUE);
209     let max = 0, at = coarse.nodes[0], j = 0;
210     for (let i = 0; i < coarse.nodes.length; i++) {
211         while (j < fine.nodes.length - 1 && fine.nodes[j] < coarse.nodes[i] - tolerance) j++;
212         if (Math.abs(fine.nodes[j] - coarse.nodes[i]) > tolerance) continue;
213         const value = Math.abs(coarse.values[i] - fine.values[j]) / denominator;
214         if (value > max) { max = value; at = coarse.nodes[i]; }
215     }
216     return { value: max, at };
217 }
218
219 // Подбор шага по правилу Рунге:  $h$  делится пополам, пока  $R > \epsilon$  на всём интервале.
220 export function refineByRunge(method, order, problem) {
221     const { equation, x0, y0, xn, epsilon } = problem;
222     const steps = [];
223     let h = problem.h;
224     let coarse;
225     try {
226         coarse = method(equation.f, x0, y0, xn, h);
227     } catch (error) {
228         return { steps, h, converged: false, reason: error.message };
229     }
230     for (let attempt = 0; attempt <= MAX_REFINEMENTS; attempt++) {
231         if ((xn - x0) / h > MAX_NODES / 2) {
232             return { steps, h, converged: false, solution: coarse, reason: "Достигнут предел $MAX_NODES узлов." };
233         }
234         let fine;
235         try {

```

```

235     fine = method(equation.f, x0, y0, xn, h / 2);
236   } catch (error) {
237     return { steps, h, converged: false, solution: coarse, reason: error.message };
238   }
239   const runge = rungeError(coarse, fine, order);
240   steps.push({ h, nodes: coarse.nodes.length, error: runge.value, at: runge.at, ok:
runge.value <= epsilon });
241   if (runge.value <= epsilon) return { steps, h, converged: true, solution: coarse,
fine, runge };
242   coarse = fine;
243   h /= 2;
244 }
245 return { steps, h, converged: false, solution: coarse, reason: Точность` не достигнута
за ${MAX_REFINEMENTS} делений шага.` };
246 }
247
248 // Погрешность по точному решению:  $\epsilon = \max |y_i \text{ точн} - y_i|$ .
249 export function exactError(nodes, values, exact) {
250   let max = 0, at = nodes[0];
251   for (let i = 0; i < nodes.length; i++) {
252     const reference = exact(nodes[i]);
253     if (!Number.isFinite(reference)) continue;
254     const value = Math.abs(reference - values[i]);
255     if (value > max) { max = value; at = nodes[i]; }
256   }
257   return { value: max, at };
258 }
259
260 // ----- Сводное решение -----
261
262 export const METHODS = [
263   { id: "euler", label: "Метод Эйлера", short: "Эйлер", kind: "one-step", order: 1, run:
euler },
264   { id: "rk4", label: "Метод РунгеКутты- го4- порядка", short: "РунгеКутта-", kind: "one-
step", order: 4, run: rungeKutta4 },
265   { id: "adams", label: "Метод Адамса предикторкорректор(-)", short: "Адамс", kind: "
multistep", order: 4, run: adams },
266 ];
267
268 export const VARIANT19_METHODS = ["euler", "rk4", "adams"];
269
270 export function solve(input) {
271   const problem = validateProblem(input);
272   const { equation, x0, y0, xn, h, epsilon } = problem;
273   const exact = equation.exact(x0, y0);
274   const nodes = gridNodes(x0, xn, h);
275   const exactValues = nodes.map(exact);
276   const singular = exactValues.some(value => !Number.isFinite(value));
277   const warnings = [];
278   if (singular) warnings.push("Точное решение имеет особенность внутри интервала:

```

```

сравнение в этих узлах невозможно.");
279 if (nodes.length < 5) warnings.push("Для метода Адамса нужно не менее четырёх шагов;
при текущем h он недоступен.");
280
281 const results = METHODS.map(method => {
282   const base = { id: method.id, label: method.label, short: method.short, kind: method.
kind, order: method.order };
283   let solution;
284   try {
285     solution = method.id === "adams"
286       ? method.run(equation.f, x0, y0, xn, h, epsilon)
287       : method.run(equation.f, x0, y0, xn, h);
288   } catch (error) {
289     return { ...base, applicable: false, reason: error.message };
290   }
291   const error = exactError(solution.nodes, solution.values, exact);
292   const item = { ...base, applicable: true, values: solution.values, error, corrections
: solution.corrections };
293   if (method.kind === "one-step") item.runge = refineByRunge(method.run, method.order,
problem);
294   return item;
295 });
296
297 const spread = (() => {
298   const applicable = results.filter(r => r.applicable);
299   if (applicable.length < 2) return 0;
300   return Math.max(...nodes.map((_, i) => {
301     const column = applicable.map(r => r.values[i]).filter(Number.isFinite);
302     return column.length < 2 ? 0 : Math.max(...column) - Math.min(...column);
303   }));
304 })();
305 if (results.some(r => r.applicable && r.error.value > 0.1 * Math.max(1, Math.abs(y0))))
{
306   warnings.push("Погрешность на текущей сетке велика: уменьшите шаг h.");
307 }
308
309 return { problem, equation, exact, nodes, exactValues, singular, results, spread,
warnings };
310 }

```

## 5 Результаты выполнения программы

### 5.1 Задача варианта

Основной набор:  $y' = y + (1 + x)y^2$ ,  $y(1) = -1$ ,  $[1; 1,5]$ ,  $h = 0,1$ ,  $\varepsilon = 10^{-6}$ . Точное решение  $y = -1/x$ .

Таблица 3: Таблица приближённых значений интеграла дифференциального уравнения

| $i$ | $x_i$ | Эйлер     | Рунге–Кутта | Адамс     | $y_{\text{точн}}$ |
|-----|-------|-----------|-------------|-----------|-------------------|
| 0   | 1,0   | −1,000000 | −1,000000   | −1,000000 | −1,000000         |
| 1   | 1,1   | −0,900000 | −0,909093   | −0,909093 | −0,909091         |
| 2   | 1,2   | −0,819900 | −0,833337   | −0,833337 | −0,833333         |
| 3   | 1,3   | −0,753998 | −0,769234   | −0,769234 | −0,769231         |
| 4   | 1,4   | −0,698640 | −0,714289   | −0,714282 | −0,714286         |
| 5   | 1,5   | −0,651360 | −0,666670   | −0,666659 | −0,666667         |

Таблица 4: Фактическая погрешность и её оценка

| Метод       | $p$ | Способ оценки                  | Оценка                                            | $\max  y_{\text{точн}} - y_i $ |
|-------------|-----|--------------------------------|---------------------------------------------------|--------------------------------|
| Эйлера      | 1   | правило Рунге                  | $8,57 \cdot 10^{-7}$ при $h = 1,22 \cdot 10^{-5}$ | $1,56 \cdot 10^{-2}$           |
| Рунге–Кутта | 4   | правило Рунге                  | $2,34 \cdot 10^{-7}$ при $h = 0,1$                | $3,72 \cdot 10^{-6}$           |
| Адамса      | 4   | $\max  y_{\text{точн}} - y_i $ | $7,51 \cdot 10^{-6}$ при $h = 0,1$                | $7,51 \cdot 10^{-6}$           |

Столбцы «Оценка» и  $\max |y_{\text{точн}} - y_i|$  у метода Эйлера относятся к разным шагам: оценка получена после автоматического дробления шага, фактическая погрешность — на исходной сетке  $h = 0,1$ .

Значения метода Эйлера и метода Рунге–Кутта совпадают с таблицами примеров 1 и 3 лекции во всех приведённых там знаках; это совпадение закреплено автоматическими тестами.

Таблица 5: Сравнение с таблицей примера 1 (метод Эйлера) и примера 3 лекции

| $x_i$ | Эйлер, лекция | Эйлер, программа | Рунге–Кутта, лекция | Рунге–Кутта, программа |
|-------|---------------|------------------|---------------------|------------------------|
| 1,1   | −0,900000     | −0,900000        | −0,909093           | −0,909093              |
| 1,2   | −0,819900     | −0,819900        | −0,833336           | −0,833337              |
| 1,3   | −0,753998     | −0,753998        | −0,769234           | −0,769234              |
| 1,4   | −0,698640     | −0,698640        | −0,714289           | −0,714289              |
| 1,5   | −0,651361     | −0,651360        | −0,666670           | −0,666670              |

Расхождения в последнем знаке объясняются округлением промежуточных значений в лекции: программа хранит все значения в двойной точности.



## 5.2 Правило Рунге

Для метода Рунге–Кутты заданная точность  $\varepsilon = 10^{-6}$  достигается сразу на исходном шаге:  $R = 2,34 \cdot 10^{-7}$ . Методу Эйлера потребовалось 14 дроблений шага.

Таблица 6: Контроль точности метода Эйлера по правилу Рунге ( $p = 1$ )

| $h$                    | Узлов | $R =  y_h - y_{h/2} /(2^p - 1)$ | $R \leq \varepsilon$ |
|------------------------|-------|---------------------------------|----------------------|
| 0,1                    | 6     | $8,27 \cdot 10^{-3}$            | нет                  |
| 0,05                   | 11    | $3,79 \cdot 10^{-3}$            | нет                  |
| 0,025                  | 21    | $1,82 \cdot 10^{-3}$            | нет                  |
| 0,0125                 | 41    | $8,94 \cdot 10^{-4}$            | нет                  |
| $6,25 \cdot 10^{-3}$   | 81    | $4,43 \cdot 10^{-4}$            | нет                  |
| $3,125 \cdot 10^{-3}$  | 161   | $2,20 \cdot 10^{-4}$            | нет                  |
| $1,5625 \cdot 10^{-3}$ | 321   | $1,10 \cdot 10^{-4}$            | нет                  |
| $7,8125 \cdot 10^{-4}$ | 641   | $5,49 \cdot 10^{-5}$            | нет                  |
| $3,9063 \cdot 10^{-4}$ | 1281  | $2,74 \cdot 10^{-5}$            | нет                  |
| $1,9531 \cdot 10^{-4}$ | 2561  | $1,37 \cdot 10^{-5}$            | нет                  |
| $9,766 \cdot 10^{-5}$  | 5121  | $6,86 \cdot 10^{-6}$            | нет                  |
| $4,883 \cdot 10^{-5}$  | 10241 | $3,43 \cdot 10^{-6}$            | нет                  |
| $2,441 \cdot 10^{-5}$  | 20481 | $1,71 \cdot 10^{-6}$            | нет                  |
| $1,221 \cdot 10^{-5}$  | 40961 | $8,57 \cdot 10^{-7}$            | да                   |

Каждое деление шага уменьшает  $R$  примерно вдвое — это прямое подтверждение первого порядка точности метода Эйлера.

## 5.3 Порядок сходимости

Для той же задачи фактическая погрешность  $\max |y_{\text{точн}} - y_i|$  измерена при последовательном дроблении шага.

Таблица 7: Порядок сходимости методов на задаче варианта

| $h$    | Эйлер                  | отношение | Рунге–Кутта             | отношение |
|--------|------------------------|-----------|-------------------------|-----------|
| 0,1    | $1,5646 \cdot 10^{-2}$ | —         | $3,7232 \cdot 10^{-6}$  | —         |
| 0,05   | $7,3797 \cdot 10^{-3}$ | 2,12      | $2,1480 \cdot 10^{-7}$  | 17,33     |
| 0,025  | $3,5972 \cdot 10^{-3}$ | 2,05      | $1,2883 \cdot 10^{-8}$  | 16,67     |
| 0,0125 | $1,7762 \cdot 10^{-3}$ | 2,03      | $7,8814 \cdot 10^{-10}$ | 16,35     |

При уменьшении шага вдвое погрешность метода Эйлера уменьшается примерно в  $2^1 = 2$  раза, метода Рунге–Кутта — примерно в  $2^4 = 16$  раз, что соответствует заявленным порядкам точности.

## 5.4 Другие наборы исходных данных

**Набор 2:**  $y' = -2xy$ ,  $y(0) = 1$ ,  $[0; 2]$ ,  $h = 0,25$ ,  $\varepsilon = 10^{-6}$ . Точное решение  $y = e^{-x^2}$ ; шаг намеренно взят крупным, чтобы участок перегиба был пройден за несколько узлов.

Таблица 8: Решение уравнения  $y' = -2xy$  при  $h = 0,25$

| $i$ | $x_i$ | Эйлер    | Рунге–Кутта | Адамс    | $y_{\text{точн}}$ |
|-----|-------|----------|-------------|----------|-------------------|
| 0   | 0,00  | 1,000000 | 1,000000    | 1,000000 | 1,000000          |
| 2   | 0,50  | 0,875000 | 0,778797    | 0,778797 | 0,778801          |
| 4   | 1,00  | 0,410156 | 0,367935    | 0,367411 | 0,367879          |
| 6   | 1,50  | 0,076904 | 0,105709    | 0,105459 | 0,105399          |
| 8   | 2,00  | 0,002403 | 0,018711    | 0,018527 | 0,018316          |

Погрешности: Эйлер  $9,62 \cdot 10^{-2}$ , Рунге–Кутта  $4,00 \cdot 10^{-4}$ , Адамс  $4,69 \cdot 10^{-4}$ . Корректор Адамса на этом наборе выполняет от 7 до 10 итераций на шаг — крупный шаг замедляет его сходимость.

**Набор 3:**  $y' = y - x^2 + 1$ ,  $y(0) = 0,5$ ,  $[0; 2]$ ,  $h = 0,2$  — задан текстовым файлом с десятичной запятой. Точное решение  $y = (x + 1)^2 - \frac{1}{2}e^x$ . В конечной точке  $x = 2$ : Эйлер 4,865785, Рунге–Кутта 5,305363, Адамс 5,305210, точное 5,305472. Погрешности соответственно  $4,40 \cdot 10^{-1}$ ,  $1,09 \cdot 10^{-4}$  и  $2,62 \cdot 10^{-4}$ .

**Набор 4:**  $y' = -5y + 5x^2 + 2x$ ,  $y(0) = 1$ ,  $[0; 3]$ ,  $h = 0,5$ ,  $\varepsilon = 10^{-4}$ . Точное решение  $y = x^2 + e^{-5x}$  содержит быстро затухающую составляющую. Условие устойчивости явного метода Эйлера  $h < 2/5 = 0,4$  нарушено, и решение осциллирует с нарастающей амплитудой.

Таблица 9: Неустойчивость явного метода Эйлера при  $h = 0,5$

| $i$ | $x_i$ | Эйлер     | Рунге–Кутта | $y_{\text{точн}}$ |
|-----|-------|-----------|-------------|-------------------|
| 0   | 0,0   | 1,000000  | 1,000000    | 1,000000          |
| 1   | 0,5   | -1,500000 | 0,979818    | 0,332085          |
| 2   | 1,0   | 3,375000  | 1,554621    | 1,006738          |
| 3   | 1,5   | -1,562500 | 2,691018    | 2,250553          |
| 4   | 2,0   | 9,468750  | 4,367353    | 4,000045          |
| 5   | 2,5   | -2,203125 | 6,569585    | 6,250004          |
| 6   | 3,0   | 21,429688 | 9,288611    | 9,000000          |

На этом наборе метод Адамса вообще неприменим: итерации корректора расходятся, и программа сообщает об этом явно, а не выдаёт заведомо неверные числа. После уменьшения шага до  $h = 0,1$  все три метода работают устойчиво.

## 5.5 Проверка программы

Таблица 10: Наборы данных для проверки

| Набор                    | Содержание проверки                                                     | Результат                     |
|--------------------------|-------------------------------------------------------------------------|-------------------------------|
| Вариант 19               | $y' = y + (1 + x)y^2$ , $y(1) = -1$ ,<br>[1; 1,5], $h = 0,1$            | расчёт выполнен               |
| Уравнение<br>$y' = -2xy$ | [0; 2], крупный шаг $h = 0,25$                                          | расчёт выполнен               |
| Текстовый файл           | $y' = y - x^2 + 1$ , десятичная<br>запятая                              | файл прочитан                 |
| Жёсткая задача           | $y' = -5y + 5x^2 + 2x$ , $h = 0,5$                                      | Эйлер неустойчив,<br>Адамс    |
| Крупный шаг              | $h > (x_n - x_0)/4$                                                     | неприменим<br>метод Адамса    |
| Некорректный<br>набор    | $h = -0,1$ ; $x_n < x_0$ ; $\varepsilon = 0$ ;<br>неизвестное уравнение | недоступен<br>выведена ошибка |

Автоматические тесты проекта проверяют совпадение методов Эйлера и Рунге–Кутты с таблицами примеров 1 и 3 лекции, экспериментальные порядки сходимости, разгон метода Адамса, правило Рунге, чтение обоих форматов файлов и обработку некорректных данных.

## 6 Графики точного и приближённого решений

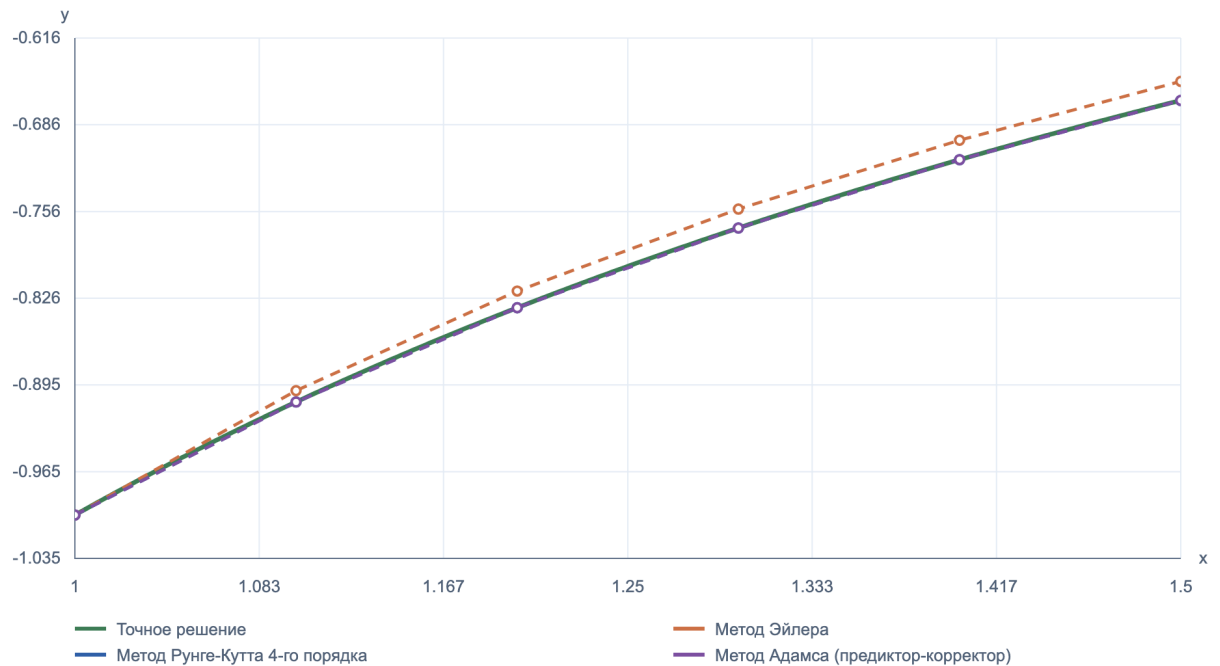


Рис. 2: Задача варианта  $y' = y + (1+x)y^2$ ,  $y(1) = -1$ ,  $h = 0,1$ : точное решение (зелёная сплошная линия) и приближённые решения (пунктир: оранжевый — Эйлер, синий — Рунге-Кутта, фиолетовый — Адамс)

Кривые Рунге-Кутта и Адамса визуально сливаются с точным решением, а ломаная метода Эйлера систематически идёт выше: на каждом шаге приращение заменяется приращением касательной, поэтому решение «соскальзывает» на соседнюю интегральную кривую.

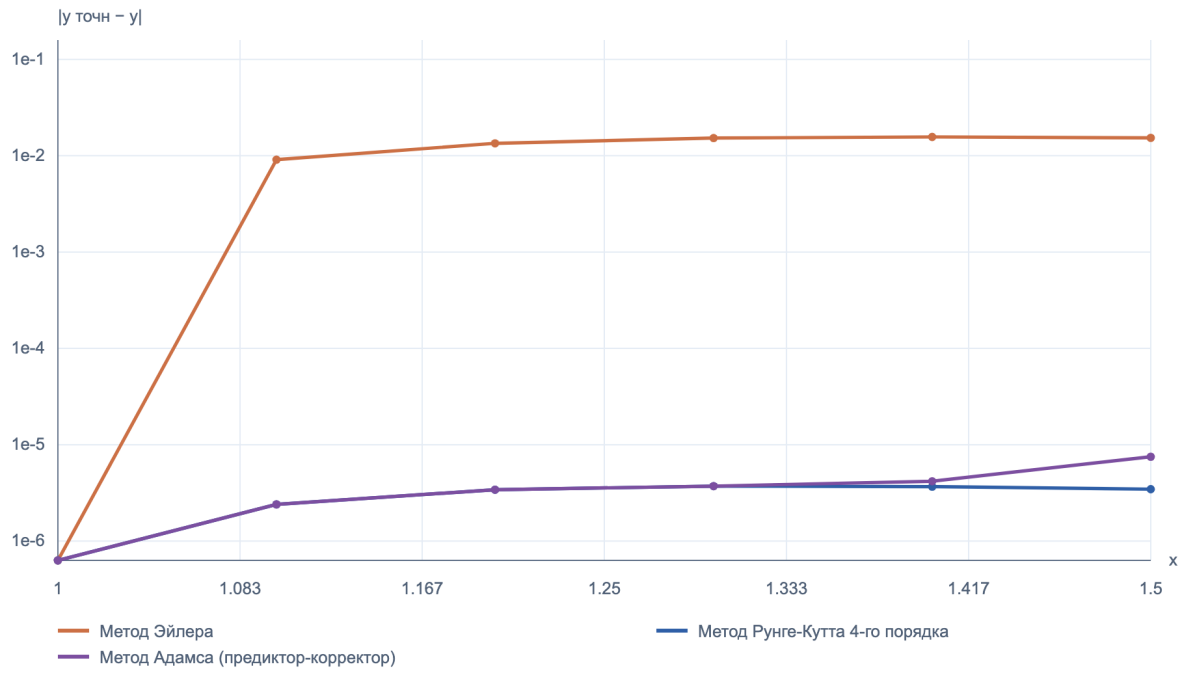


Рис. 3: Погрешность  $|y_{\text{точен}} - y_i|$  в узлах, логарифмическая шкала

График погрешности показывает разницу в три-четыре порядка. Видно и то, что метод Адамса до узла  $x = 1,3$  повторяет метод Рунге–Кутта (эти узлы получены разгоном), а дальше его собственная погрешность начинает расти быстрее.

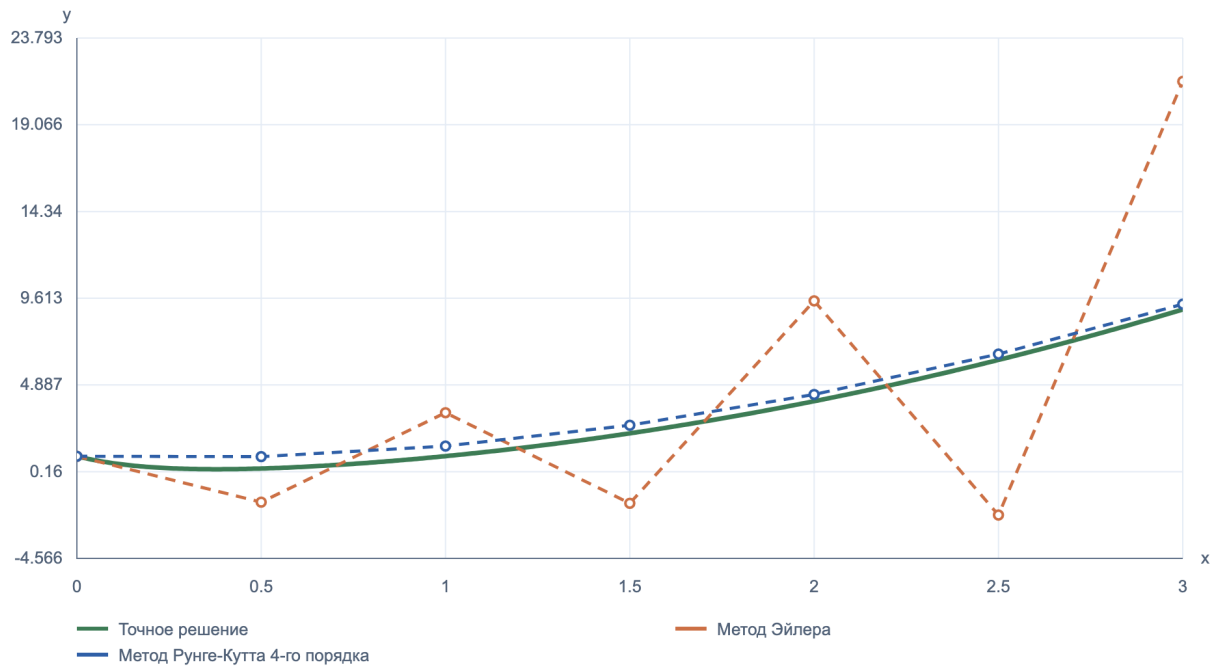


Рис. 4: Жёсткая задача  $y' = -5y + 5x^2 + 2x$  при  $h = 0,5$ : явный метод Эйлера неустойчив, метод Рунге–Кутты сохраняет качественно верное поведение

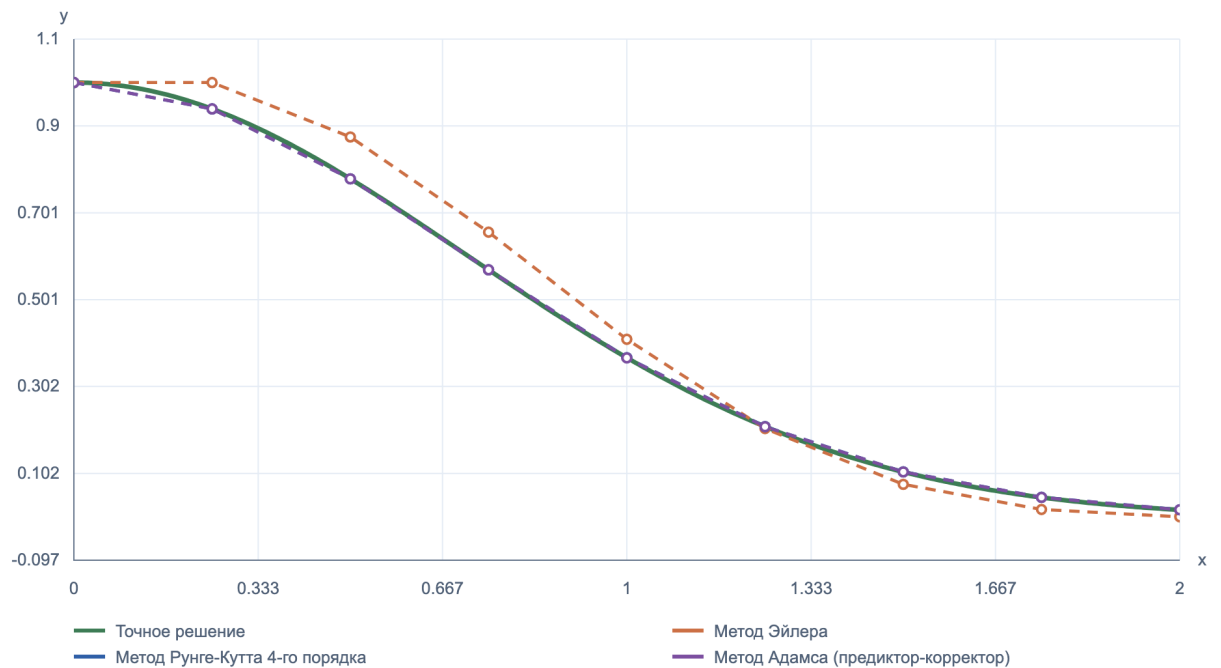


Рис. 5: Уравнение  $y' = -2xy$ ,  $y(0) = 1$ ,  $h = 0,25$ : крупный шаг заметен на участке наибольшей кривизны решения

## 7 Скриншоты результатов работы программы

Оформление интерфейса продолжает ЛР №4 и ЛР №5: светлый фон, синие акценты, левая панель ввода и правая область результатов. Ввод  $x_0, y_0, x_n, h, \varepsilon$  и выбор уравнения выполняются в одной форме; таблицы, графики и оценки погрешности обновляются по кнопке «Решить».

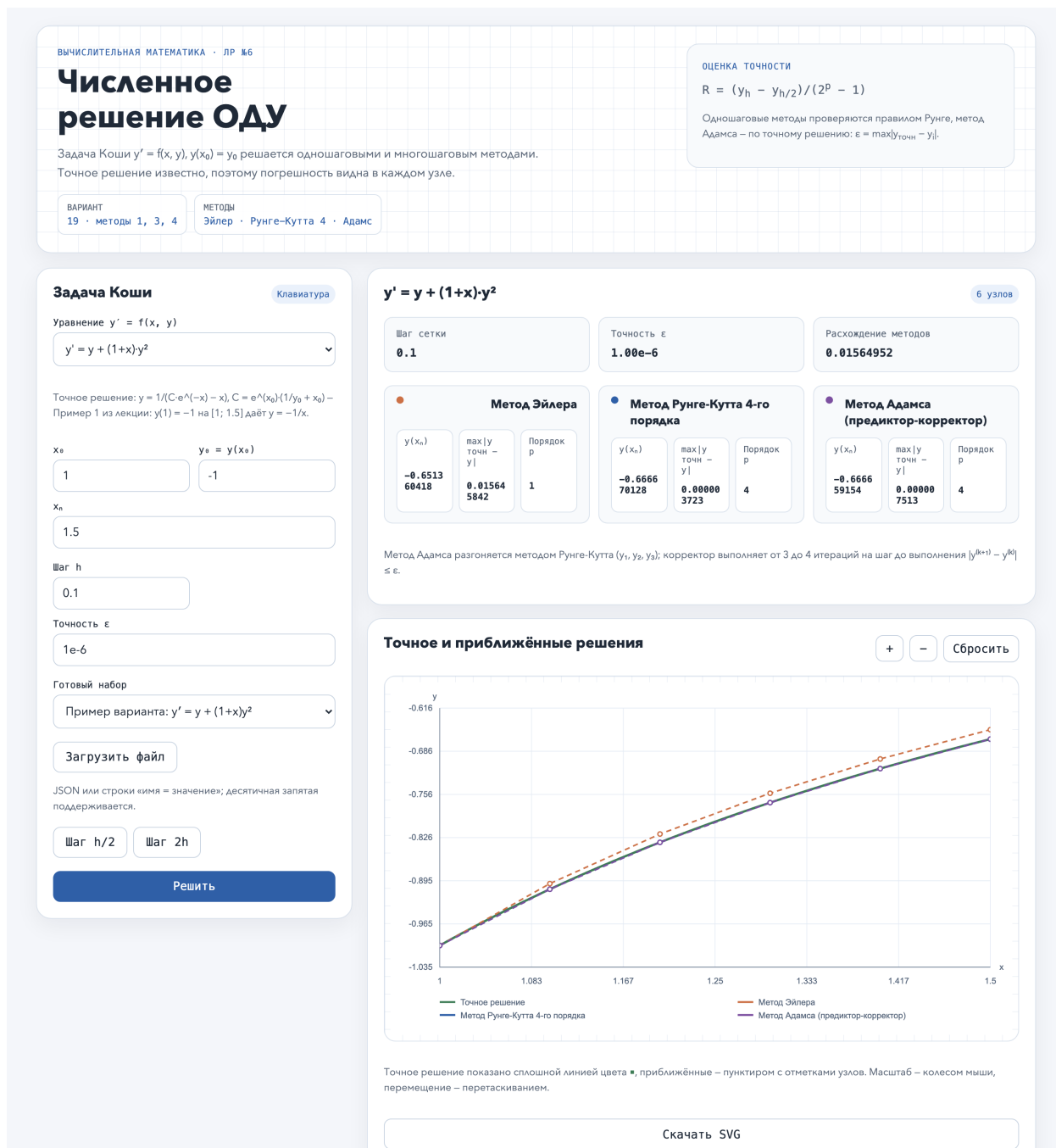


Рис. 6: Набор 1 — задача варианта: параметры, сводка по методам и график точного и приближённых решений

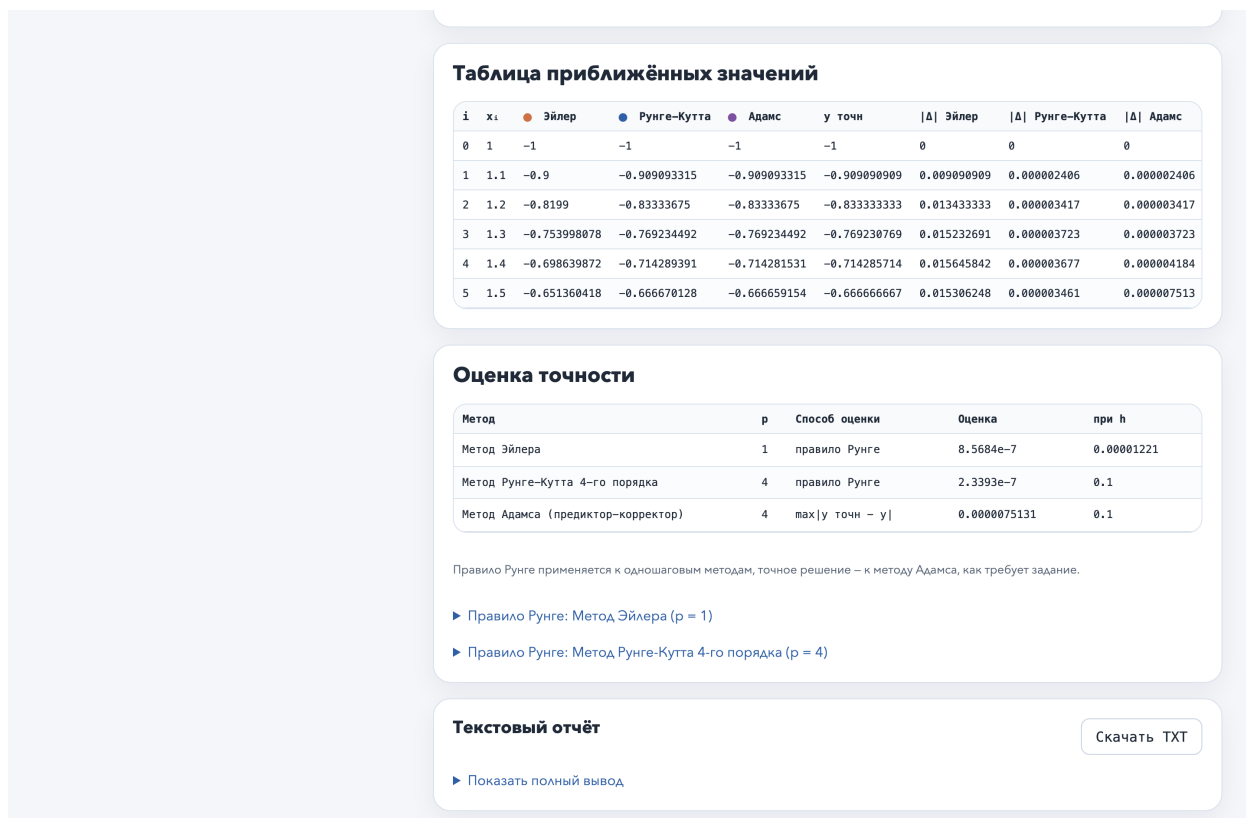


Рис. 7: Тот же запуск: таблица приближённых значений с погрешностью в каждом узле и таблица оценок точности





ВАРИАНТ  
19 · методы 1, 3, 4

МЕТОДЫ  
Эйлер · Рунге–Кутта 4 · Адамс

Клавиатура

$$y' = y + (1+x) \cdot y^2$$

| $x_0$ | $y_0 = y(x_0)$ |
|-------|----------------|
| 1     | -1             |

War h

-0.1

Точность  $\epsilon$

1e-6

Готовый набор

Пример варианта:  $y' = y + (1+x)y^2$

Загрузить файл

JSON или строки «имя = значение»; десятичная запятая поддерживается.

 $\frac{h}{2}$ 

War 2h

Решить

Шаг  $h$  должен быть положительным.

Нажмите «Решить», чтобы обновить таблицу и графики.

$$R = (y_h - y_{h/2}) / (2^p - 1)$$

Одношаговые методы проверяются правилом Рунге, метод Адамса – по точному решению:  $\varepsilon = \max|y_{\text{точн}} - y_i|$ .

24

## Численное решение ОДУ

Задача Коши  $y' = f(x, y)$ ,  $y(x_0) = y_0$  решается одношаговыми и многошаговыми методами. Точное решение известно, поэтому погрешность видна в каждом узле.

ВАРИАНТ

19 · методы 1, 3, 4

МЕТОДЫ

Эйлер · Рунге-Кутта 4 · Адамс

ОЦЕНКА ТОЧНОСТИ

$$R = (y_h - y_{h/2}) / (2^p - 1)$$

Одношаговые методы проверяются правилом Рунге, метод Адамса – по точному решению:  $\epsilon = \max |y_{\text{точн}} - y_i|$ .

### Задача Коши

Клавиатура

Уравнение  $y' = f(x, y)$

$$y' = y + (1+x) \cdot y^2$$

Точное решение:  $y = 1/(C \cdot e^{(-x)} - x)$ ,  $C = e^{(x_0)} \cdot (1/y_0 + x_0)$  – Пример 1 из лекции:  $y(1) = -1$  на  $[1; 1.5]$  даёт  $y = -1/x$ .

$x_0$

$y_0 = y(x_0)$

1

-1

$x_n$

1.5

Шаг h

Рис. 10: Адаптивная форма в окне шириной 390 пикселей

## 8 Вывод

Решена задача Коши  $y' = y + (1+x)y^2$  с начальным условием  $y(1) = -1$  на отрезке  $[1; 1.5]$  с шагом  $h = 0,1$  и точностью  $\epsilon = 10^{-6}$ ; точное решение  $y = -1/x$ . В точке  $x = 1,5$  метод Эйлера дал  $-0,651360$ , метод Рунге-Кутта 4-го порядка —

$-0,666670$ , метод Адамса —  $-0,666659$  при точном значении  $-0,666667$ . Погрешность  $\max |y_{\text{точн}} - y_i|$  равна  $1,56 \cdot 10^{-2}$  у метода Эйлера,  $3,72 \cdot 10^{-6}$  у метода Рунге–Кутта и  $7,51 \cdot 10^{-6}$  у метода Адамса; значения обоих одношаговых методов совпали с таблицами примеров 1 и 3 лекции.

При  $h = 0,05$  те же погрешности равны  $7,38 \cdot 10^{-3}$  и  $2,15 \cdot 10^{-7}$ , то есть в 2,1 и 17,3 раза меньше, чем при  $h = 0,1$ . По правилу Рунге точность  $\varepsilon = 10^{-6}$  достигнута методом Рунге–Кутта при  $h = 0,1$  (6 узлов), методом Эйлера — при  $h = 1,22 \cdot 10^{-5}$  (40961 узел). Метод Адамса начинает счёт с  $y_4$ ; значения  $y_1, y_2, y_3$  вычислены методом Рунге–Кутта, корректор выполняет 3–4 итерации на шаг. Программа выводит сообщение об ошибке при  $h \leq 0, x_n \leq x_0, \varepsilon$  вне интервала  $(0; 1)$  и нечисловом вводе.